Prac 1

Ayush Beekum

February 2026

1 Question 1) Explain what an esoteric programming language (or esolang) is.

The esoteric programming language, is a programming language created not for everyday use in the coding world, but rather something that is unique, difficult and requires critical thinking [1]. It is not for mainstream use, like java for example. It may involve creating a language that is minimal, obfuscated or mimics another language (though in an informal way) [2]. It is commonly used by hackers and people who want to explore the boundaries of theoretical computation [3].

2 Question 2) Esoteric programming languages are usually considered to have no real-world applications, and are therefore usually considered to be useless. Suggest four useful contributions that esoteric programming languages can make.

Since there are no strict restrictions on esoteric programming, this allows for no boundaries and freedom of breaking normal/expected coding practices and structures. Why could this be beneficial? It allows us to explore different ways to go about on coding something, code execution, different syntax structure. It allows for freedom away from normal coding conventions, to approach coding in an entirely different perspective to tackle a problem. One of the examples of this can be Befunge, they produced a 2D instruction pointer, taking on the linear approach of code execution and changing the perspective of how we view program flow [3].

In the first question i talked about pushing the boundaries of theoretical computation, by using this form of programming, many of these languages are

designed to be "Turing tarpits", that turing complete languages with a low number of instructions being given/inputted [1].

One of the other reasons can be the fact that since there are no conventions to stick to, no boundaries to stay within, we can encourage critical thinking by forcing the person to think about going about solving a problem in completely new ways. No need to follow this specific way or must be done that way, it is entirely up to you to create a solution to tackle a problem in your own unique way. This also reinforces the fact the we can see how data is used and manipulated, and how to turn the data into something useful to solve the problem.

The language can also be used in way to portray a form of art or humour. For example LOLCODE parodies the various memes that float around the internet, this shows that code does not have to be about building complex applications and software but also a tool for creativity and humor or art [4].

3 Question 3) Choose any two esoteric programming languages. Describe the languages in terms of their designer(s), year of initial design, and their general syntactic and semantic characteristics. Provide a short example code snippet, to illustrate the language's general characteristics (you do not have to write the code yourself).

Malbolge Malbolge, an esoteric language created by Ben Olmstead with the initial year of design 1998 [5]. The language was named after the 8th circle of Hell in Dante's Inferno, and was designed for the sole purpose of being as annoying and difficult to program as possible [6]. It involves self-altering code where things like the instruction at a specific memory address changes after being executed or using base-three arithmetic as it is a tritwise, self-modifying operation [7]. The program was so hard to code that its first program was created 2 years after it was created, and not by a human but by an algorithm, namely the beam search algorithm [3]. The code's behaviour is incredibly hard to debug due to the fact that it has 8 set of instructions, and their effects depend on the current memory addresses value modulo 3, and previously stated the address changes after every single execution [5].

Below is a code snippet:

```
(=<‘#9]~6ZY32Vx/4Rs+0No-&Jk)"Fh}|Bcy?‘=*z]Kw%oG4UUS0/@-ejc(:’8dc
```

Shakespeare The second language is called Shakespeare designed by Jon Åslund and Karl Hasselström in 2001 [8]. The idea behind this language is to make source code resemble a Shakespearean play, sort of like a script [4]. Variables are names of characters in the play ("Macbeth" for example). The values are changed by the dialogue thats outputted, for example one characters line is an instruction to change the other character's value [1]. Character names have positive and negative numerical values and stacks are in this language for function calls. It also has fully defined semantics and syntax, even a parser that validates the play structure [3].

Below is a code snippet:

```
The Ghost:
    You are as villainous as the square root of 2!

Act I: Hamlet's insults and flattery.
```

4 Question 4

Consider the programming language SALT. Briefly explain where SALT was developed, what type of programming language SALT is, and what advantage it provides. Also suggest two real-world programming domains that would greatly benefit from its use.

Origin SALT was originated and developed at the University of Lübeck's Institute for Software Engineering and Programming Languages in Germany [9].

Type of programming language SALT programming language is a domain-specific specification language used specifically for temporal logic [9].

The advantages of SALT There are several different advantages of using SALT, but one of the main reasons is that it makes formal specification more accessible to software engineers [9]. SALT, in a way, acts like a "front-end", allowing engineers to write specifications in a more structured way, and from here a translator converts these specifications into LTL formulae (LTL being Linear Temporal Logic) that can then be used as input for model checkers or runtime verification tools. Thus bridging the gap between practical software development and formal methods [9].

Real-world programming domains that are beneficial 1. Concurrent and distributed systems: cloud-based services are becoming increasingly popular and more and more frequently used around the world. The components of these services must interact in a time-sensitive and orderly manner in order to avoid data corruption or deadlock. This is where SALT comes in, as SALT could be used to specify the correct ordering of events, like a request must always be

followed by a reply, or no two requests in a row without a reply being in between those requests.

2. Safety-Critical embedded system: lets take a look at modern vehicles, every vehicle comes with an Engine Control Unit (ECU) that monitor specific parts of the vehicle like temperature, transmission status, brake systems. It is important that these parts are in a condition that is not threatening to the vehicle (being in bad condition or compromised) and if they are then to notify the user of the vehicle. For example with SALT, a specification could be that a message needs to display on the cluster of the vehicle if the tyre pressure of any of the tyres are below the required amount.

5 Question 5

Briefly explain what "vibe coding" entails, and briefly describe two potential advantages and two potential drawbacks to this practice.

What is vibe coding? Vibe coding is essentially a term created by Andrej Karpathy that is based on a new approach in the modern world involving using artificial intelligence in software development [10]. To quote it is the practice of "fully giving in to the vibes, embracing exponential, and forgetting that the code even exists." It works basically by the developer describing what they want in a natural language, and an AI (like chatGPT, deepseek , claude) to generate the code. So the developer's role is no longer writing code but just consistently telling the AI to build the code and fix any errors that come along the way or things to change or to quote Karpathy, " just see stuff, say stuff, run stuff, and copy-paste stuff, and it mostly works" [10].

2 advantages of AI 1. Coding is well known to appear as intimidating and difficult to understand and get into, but because of the introduction to AI this barrier has been lowered significantly as it has made creation of software relatively easier [10]. Instead of spending hours on learning code to translate your idea into something real you can use AI tools to speed up this process significantly without needing to have a strenuous amount of experience in developing software. This allows for faster development of ideas and quick iterations of concepts.

2. Some parts of code are simple to make and yet requires alot of time to implement, for example basic testing. AI can accelerate this process by completing the simple and easy tedious task in seconds so you can focus more of your time on tackling the more in depth and complex parts of a program/software [10].

2 disadvantages of AI 1. From personal experience, yes AI can be incredibly useful to complete programs, it is not perfect in the creation of it. Frequent bugs start to appear, over complicated solutions and the issue of not actually learning anything. AI may be useful to complete your tasks, but also takes away the fact that you do not learn anything if the ai does everything for you, not to

mention the fact that if the code does not work and you made AI do all the work, how will one be able to fix it if they have just let AI do all the work for them? That is where the real issue arises, the ability to learn and understand code independently is taken away when one is reliant on "Vibe coding" and if an issue ensues how will you fix it if you don't know what is happening within the code? This is a serious problem as yes we have AI to build our programs but they struggle at complex algorithms and it is up to us to be able to fix the code.

2. Partly mentioned in the first point, is the fact that bugs, security vulnerabilities and maintainability issues start to arise more and more frequently as programs get more and more complex. Vibe coders may not understand the coding logic completely raising serious difficulty in debugging issues that may occur.

References

[1] Esoteric programming language. URL: https://en.wikipedia.org/w/index.php?title=Esoteric_programming_language&printable=yes.

[2] Esoteric programming language. URL: https://ipfs.io/ipfs/bafybeiemxf5abjwjbikoz4mc3a3dla6ual3jsgpdr4cjr3oz3evfyavhwq/wiki/Esoteric_programming.html.

[3] Esoteric programming language. URL: http://wayback.archive-it.org/all/20111013014507/http://en.wikipedia.org/wiki/Esoteric_programming_language.

[4] Esoteric programming language. URL: https://en.wikipedia.org/wiki/Chef_(programming_language).

[5] Esoterische programmiersprache. URL: https://de.wikipedia.org/wiki/Esoterische_Programmiersprache.

[6] Malbolge. URL: https://graphsearch.epfl.ch/en/concept/237720.

[7] Esoteric programming language. URL: https://en.wikipedia.org/w/index.php?diff=925565314.

[8] awesome-esolangs. URL: https://awesome.ecosyste.ms/lists/angrykoala%2Fawesome-esolangs.

[9] Salt – structured assertion language for temporal logic. URL: https://www.isp.uni-luebeck.de/salt.

[10] M. Hunger. From misconceptions to momentum – the case for vibe coding in modern enterprises. URL: https://www.techradar.com/pro/from-misconceptions-to-momentum-the-case-for-vibe-coding-in-modern-enterprises.